

Simulador de Gás Ideal

Octree x KDTree

Computação Gráfica e Visualização
Daniel Couto

Sumário

- Introdução
 - Desafio Computacional
 - Cenário 3D
 - Octree
 - KDTree
 - Comparação
-

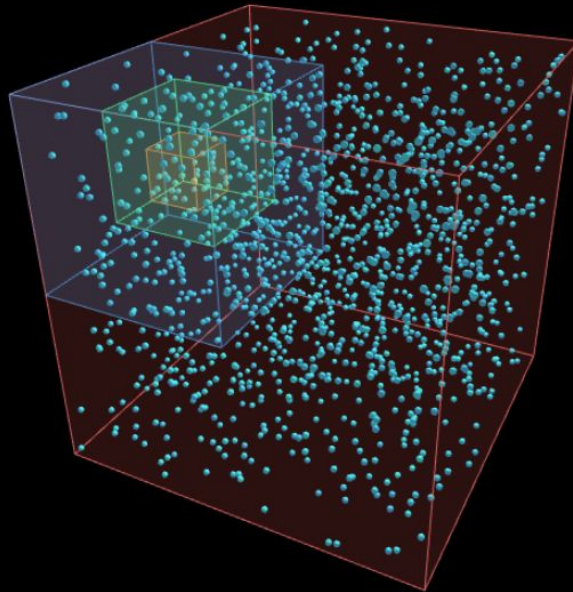
Simulação

Laboratório de Termodinâmica: Gás Ideal 3D

TERMÔMETRO (T)
100 K

PRESSURIZADOR (P)
0.18 atm

VOLUME (V)
1065 L

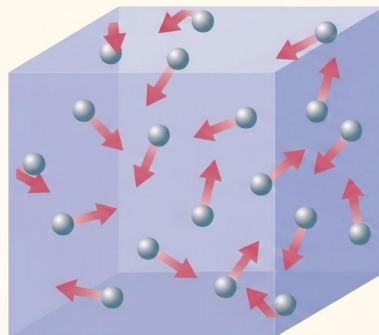


Introdução

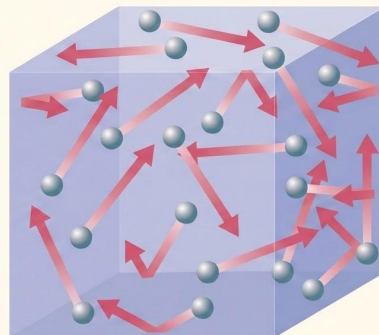
O simulador de gás ideal (em processos adiabáticos)

O simulador modela o comportamento dinâmico de um gás ideal através da interação microscópica de partículas em movimento caótico. Ele integra as leis clássicas da termodinâmica com a teoria cinética para calcular pressão via colisões (perfeitamente elásticas).

$$\text{Lei dos Gases Ideais}$$
$$PV = nRT$$



Menor temperatura



Maior temperatura

Desafios da implementação



Desafios

Ideia:

1. Iniciar as moléculas com posições e velocidades aleatórias.
2. Calcular as colisões em tempo real entre as moléculas e com as paredes.
3. Calcular a pressão de acordo com a média de colisões por segundo com as paredes.

Problema:

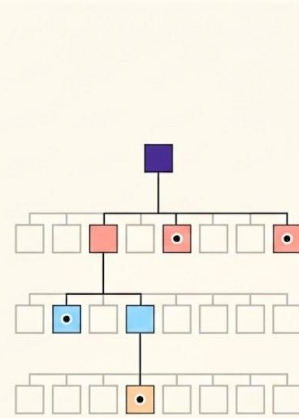
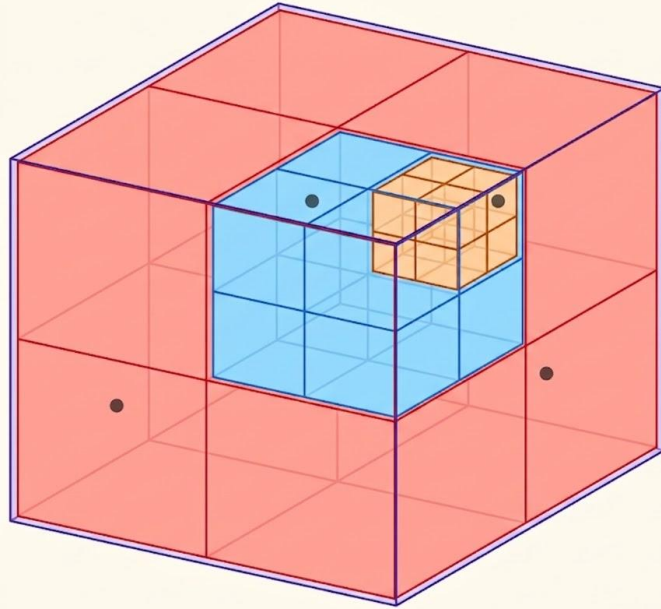
Se implementarmos de maneira ingênua, a verificação de colisão testa todas as possíveis combinações de moléculas possíveis ($O(n^2)$). Para $n = 1000$, temos mais de 1.000.000 checagens por frame, destruindo a performance do simulador no navegador.

Cenário 3D

O Pipeline Gráfico: WebGL, Cena e Renderizador

A transição para o ambiente 3D substitui o canvas 2D nativo por uma infraestrutura baseada em **WebGL**, permitindo renderização de alta performance acelerada por hardware (GPU).

- **WebGLRenderer:** Gerencia o contexto gráfico, buffers de profundidade (Z-buffer) e executa a rasterização.
- **Scene:** Grafo de cena hierárquico que gerencia todos os objetos visíveis (esferas das partículas, caixas delimitadoras e luzes).
- **Canvas HTML:** Elemento do DOM onde a imagem final calculada pela GPU é composta e exibida ao usuário.



Como funciona:

1. **Particionamento Espacial:** A caixa é fatiada recursivamente em **8 cubos** iguais sempre que uma região atinge o limite máximo de partículas.
2. **Filtragem por Região:** Cada partícula cria um volume de varredura ao seu redor e consulta a árvore para ignorar todas as moléculas que estão em octantes distantes.
3. **Cálculo de Impacto:** Aplicamos o Teorema de Pitágoras apenas entre o grupo reduzido de partículas vizinhas que restaram.

Implementação Octree

```
class QuadTree {
  constructor(boundary, capacity) {
    this.boundary = boundary;
    this.capacity = capacity;
    this.particles = [];
    this.divided = false;
  }

  subdivide() {
    const x = this.boundary.x;
    const y = this.boundary.y;
    const w = this.boundary.w / 2;
    const h = this.boundary.h / 2;

    this.ne = new QuadTree(new Rectangle(x + w, y - h, w, h), this.capacity);
    this.nw = new QuadTree(new Rectangle(x - w, y - h, w, h), this.capacity);
    this.se = new QuadTree(new Rectangle(x + w, y + h, w, h), this.capacity);
    this.sw = new QuadTree(new Rectangle(x - w, y + h, w, h), this.capacity);
    this.divided = true;
  }
}
```

```

class Octree {
    constructor(minX, maxX, minY, maxY, minZ, maxZ, capacity) {
        this.minX = minX; this.maxX = maxX;
        this.minY = minY; this.maxY = maxY;
        this.minZ = minZ; this.maxZ = maxZ;
        this.capacity = capacity;
        this.pts = [];
        this.children = null;
    }

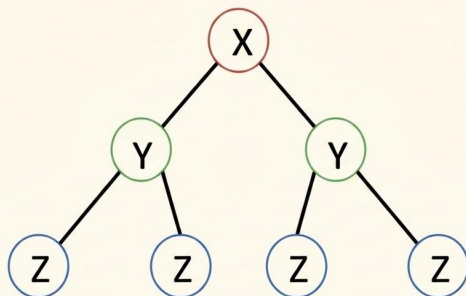
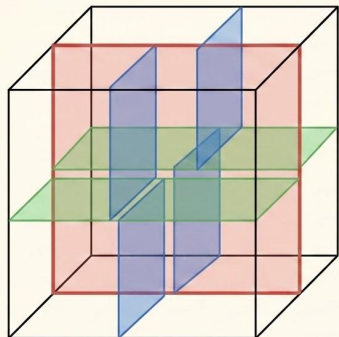
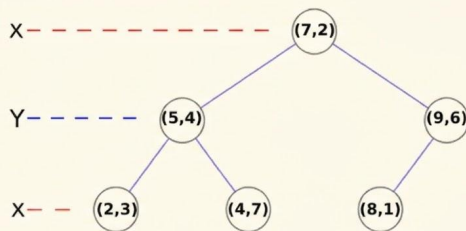
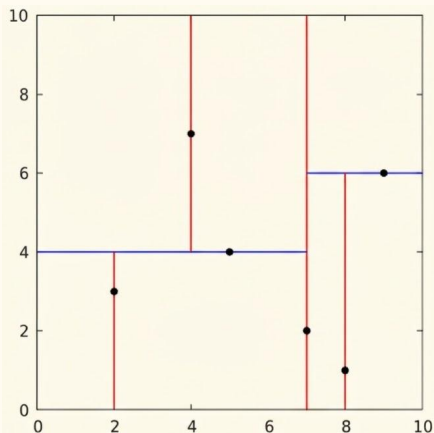
    // Cria exatamente 8 octantes e redistribui as partículas existentes.
    subdivide() {
        const mx = (this.minX + this.maxX) / 2;
        const my = (this.minY + this.maxY) / 2;
        const mz = (this.minZ + this.maxZ) / 2;
        const c = this.capacity;

        this.children = [
            new Octree(this.minX, mx, this.minY, my, this.minZ, mz, c), // ---
            new Octree(mx, this.maxX, this.minY, my, this.minZ, mz, c), // +--
            new Octree(this.minX, mx, my, this.maxY, this.minZ, mz, c), // -+-
            new Octree(mx, this.maxX, my, this.maxY, this.minZ, mz, c), // ++-
            new Octree(this.minX, mx, this.minY, my, mz, this.maxZ, c), // --+
            new Octree(mx, this.maxX, this.minY, my, mz, this.maxZ, c), // +-+
            new Octree(this.minX, mx, my, this.maxY, mz, this.maxZ, c), // -++
            new Octree(mx, this.maxX, my, this.maxY, mz, this.maxZ, c), // +++
        ];

        for (const p of this.pts) {
            for (const child of this.children) {
                if (child.insert(p)) break;
            }
        }
        this.pts = [];
    }
}

```

KD-Tree



Como funciona:

1. **Divisão Binária Adaptativa:** A árvore realiza cortes binários alternando os eixos **x**, **y** e **z**, utilizando a partícula mediana como pivô para garantir um balanceamento perfeito do espaço.
2. **Descarte por Eliminação Binária:** Durante a busca, o algoritmo descarta instantaneamente metade do espaço a cada nível da árvore, ignorando regiões que não interceptam o raio de alcance da partícula.
3. **Isolamento Logarítmico:** O processo localiza vizinhos geográficos em tempo $O(\log n)$, reduzindo drasticamente a carga de processamento ao focar apenas nas colisões fisicamente prováveis.

Implementação KD-Tree

```
// KD-TREE
class KNode {
    constructor(particle, axis, minX, maxX, minY, maxY) {
        this.particle = particle;
        this.axis = axis;
        this.left = null;
        this.right = null;
        this.minX = minX;
        this.maxX = maxX;
        this.minY = minY;
        this.maxY = maxY;
    }
}
```

```

class KDTree {
  constructor() {
    this.root = null;
  }

  build(particlesList) {
    this.root = this._buildRecursive(particlesList, 0, 0, boxWidth, 0, boxHeight);
  }

  _buildRecursive(list, depth, minX, maxX, minY, maxY) {
    if (list.length === 0) return null;

    const axis = depth % 2;

    if (axis === 0) {
      list.sort((a, b) => a.x - b.x);
    } else {
      list.sort((a, b) => a.y - b.y);
    }

    const medianIndex = Math.floor(list.length / 2);
    const p = list[medianIndex];
    const node = new KDNode(p, axis, minX, maxX, minY, maxY);

    if (axis === 0) {
      node.left = this._buildRecursive(list.slice(0, medianIndex), depth + 1, minX, p.x, minY, maxY);
      node.right = this._buildRecursive(list.slice(medianIndex + 1), depth + 1, p.x, maxX, minY, maxY);
    } else {
      node.left = this._buildRecursive(list.slice(0, medianIndex), depth + 1, minX, maxX, minY, p.y);
      node.right = this._buildRecursive(list.slice(medianIndex + 1), depth + 1, minX, maxX, p.y, maxY);
    }

    return node;
  }
}

```



```

class KDTree3D {
    constructor() {
        this.root      = null;
        this.nodeCount = 0;
    }

    build(particlesList) {
        this.nodeCount = 0;
        this.root = this._buildNode(particlesList.slice(), 0);
    }

    buildNode(pts, depth) {
        if (pts.length === 0) return null;
        this.nodeCount++;

        // Alterna o eixo de divisão a cada nível: x → y → z → x → ...
        const axis = depth % 3;
        const key  = ['x', 'y', 'z'][axis];

        pts.sort((a, b) => a[key] - b[key]);
        const mid = Math.floor(pts.length / 2);

        return {
            p:      pts[mid],
            axis,
            left:   this._buildNode(pts.slice(0, mid),      depth + 1),
            right:  this._buildNode(pts.slice(mid + 1),      depth + 1),
        };
    }
}

```

Colisões

```

function resolveCollision(p1, p2) {
    const dx = p2.x - p1.x;
    const dy = p2.y - p1.y;
    const dz = p2.z - p1.z;
    const distSq = dx*dx + dy*dy + dz*dz;
    const minDist = p1.r + p2.r;
    if (distSq >= minDist * minDist) return;

    let dist = Math.sqrt(distSq);
    if (dist === 0) dist = 1; // evita divisão por zero em sobreposição total

    const nx = dx / dist, ny = dy / dist, nz = dz / dist;

    // Separação geométrica – empurra as esferas para fora da sobreposição
    const overlap = minDist - dist;
    p1.x -= nx * overlap * 0.5; p1.y -= ny * overlap * 0.5; p1.z -= nz * overlap * 0.5;
    p2.x += nx * overlap * 0.5; p2.y += ny * overlap * 0.5; p2.z += nz * overlap * 0.5;

    // Impulso elástico 3D – conserva momento linear e energia cinética (massas iguais)
    const kx = p1.vx - p2.vx, ky = p1.vy - p2.vy, kz = p1.vz - p2.vz;
    const impulse = nx*kx + ny*ky + nz*kz;
    p1.vx -= impulse * nx; p1.vy -= impulse * ny; p1.vz -= impulse * nz;
    p2.vx += impulse * nx; p2.vy += impulse * ny; p2.vz += impulse * nz;
}

```

Comparação

Referências:

- PHET INTERACTIVE SIMULATIONS. Propriedades dos Gases. University of Colorado Boulder. https://phet.colorado.edu/pt_BR/simulations/gas-properties
- SHIFFMAN, Daniel. Quadtree - Part 1. YouTube (The Coding Train), 2018. Disponível em: youtu.be/Rf3wl1UA9Uk.
- WIKIPÉDIA. Árvore k-d. Wikipedia, a enciclopédia livre. Disponível em: pt.wikipedia.org/wiki/Árvore_k-d.
- WIKIPEDIA. Octree. Wikipedia, the free encyclopedia. Disponível em: en.wikipedia.org/wiki/Octree.

Obrigado!